From research code to reproducible infrastructure with Galaxy user-defined tools

Marius van den Beek[1], **John Chilton**[1], **Dannon Baker**[2] [1]Penn State University · [2]Johns Hopkins University

Galaxy has thousands of tools, yet the one step you need is often missing. A UDT closes that **last mile**: write it, test it on real data on a production server, iterate, then **graduate** it to a community tool.

Try it on usegalaxy.org
Request access once at udt-signup.galaxyproject.org. No install or review per tool.

A The gap

Research code starts as a notebook, a lab script, a published method or AI-drafted code. Writing the first version is the easy part. To become a step you can rerun, scale up and share, it also needs:

1. **Interface** — typed inputs and outputs
2. **Environment** — the exact software
3. **Data** — large datasets, in place
4. **Compute** — cores, memory, GPUs
5. **Provenance** — what ran, on what, how
6. **Sharing** — a step others can reuse

On a shared Galaxy server, a new tool must also be wrapped, reviewed and installed by an administrator: the right default for shared tools, but slow for one small step.

A user-defined tool (UDT) covers all six in about 40 lines, in your own account, with no install to wait for.

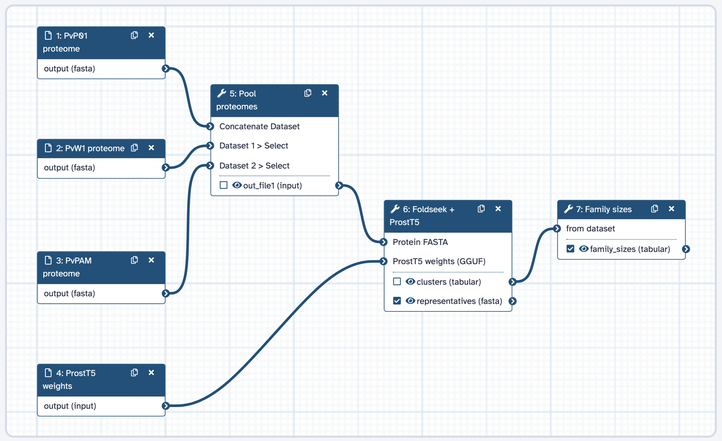
B The last mile

3 Data · 6 Sharing

usegalaxy.org runs **~2,500 tools** from a Tool Shed of **8,000+ repositories**. What's missing is usually one small step: a new method, an extra option, a wrapper merged but not yet installed.

Write → **Run on real data** → **Inspect** → **Edit** ↻

Each round is a save in the tool editor, on the production server and real data.



Three *P. vivax* proteomes (PvP01, PvW1, PvPAM) are pooled, clustered by the Foldseek UDT (step 6), and counted into family sizes with standard tools. Importing the workflow brings the UDT along.

Then graduate it to a community tool

UDT in your account → **tools-iwc-lab** → **tools-iuc**

When a UDT proves itself, turn it into a regular wrapper and open a pull request to **galaxyproject/tools-iwc-lab**: Planemo lint and tests in CI, published to the Tool Shed under the experimental `iwc-lab` owner. Mature wrappers move on to tools-iuc, installable on any Galaxy server.

C Anatomy of a UDT

Foldseek clustering with ProstT5-predicted structures, as run on usegalaxy.org with 32 cores and a GPU, in Galaxy's tool editor.

Tool Editor

```
1  class: GalaxyUserTool
2  id: foldseek_prostt5_cluster
3  name: Foldseek structural clustering
4  version: 1.0.0
5  description: Cluster proteins by predicted structure on a GPU
6  container: quay.io/biocontainers/foldseek:10.941cd33                  1
7  requirements:                                                         2
8    - type: resource
9      cores_min: 32
10     ram_min: 8192
11     cuda_device_count_min: 1
12 shell_command: |
13   mkdir weights && cp '$(inputs.model.path)' \                         3
14     weights/prostt5-f16.gguf
15   foldseek easy-cluster '$(inputs.seqs.pa
16     --prostt5-model weights --gpu 1 \                                 4
17     --threads "$GALAXY_SLOTS" \
18     -c $(inputs.coverage) -e $(inputs.evalue)                         5
19 inputs:
20   - name: seqs
21     label: Protein FASTA
22     type: data
23     format:
24       - fasta
25   - name: model
26     label: ProstT5 weights (GGUF)
27     type: data
28   - name: coverage
29     label: Minimum alignment coverage
30     type: float
31     value: 0.8
32   - name: evalue
33     label: E-value threshold
34     type: float
35     value: 0.001
36 outputs:                                                              6
37   - name: clusters
38     type: data
39     format: tabular
40     from_work_dir: clu_cluster.tsv
41   - name: representatives
42     type: data
43     format: fasta
44     from_work_dir: clu_rep_seq.fasta
```

(property) evalue: number
E-value threshold

1. The container is the environment, pinned by tag
2. A resource request: cores, memory, a GPU (**E**)
3. `$(inputs.…)` are typed, sandboxed expressions; hover shows each type (**D**)
4. `$GALAXY_SLOTS`: the site sets the core count
5. Typed inputs become the form in **F**
6. Outputs become datasets with provenance

D Safe by construction

In the tool definition

- Commands are templated with sandboxed JavaScript expressions that cannot reach the database or filesystem
- A container is mandatory
- Inputs are typed and validated before a job exists

At the site

- Access is granted once per account
- Routing and resource limits stay with the site (**E**)

> Each job runs with minimal permissions and can only access its own input and output data.

E Requesting compute

The tool declares what it needs

```
cores_min: 32                → cores 32
ram_min: 8192                → mem 8 GiB
cuda_device_count_min: 1     → gpus 1
```

The site's job router (TPV) applies policy

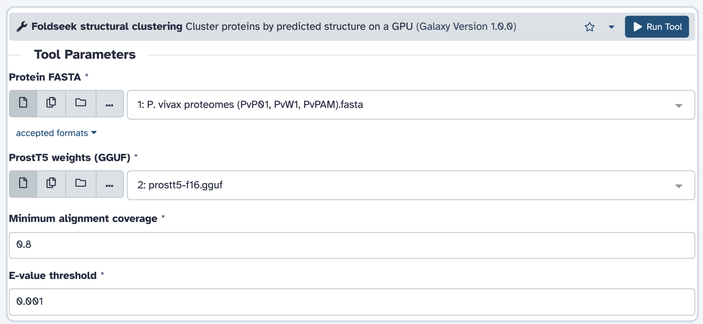
- Only destinations that opt in accept UDT jobs
- GPU requests need the **GPU Access** role
- Per-user pools cap a user's total cores, memory, GPUs

Jetstream2 GPU node (NSF ACCESS)

- Pulsar + Singularity; container required
- 32 cores become $GALAXY_SLOTS; 1 GPU

F What Galaxy does with it

Validate → **Build form** → **Run in container** → **Record provenance** → **Reuse in workflows**



The form Galaxy built from the `inputs` block: dataset pickers filtered by format, labelled numeric fields with defaults.

G What AI agents do with UDTs

- **Galaxy Workflow Foundry** gives AI agents the knowledge to translate pipelines from other systems into Galaxy workflows. Converting the Nextflow pipeline nf-core/rnaseq for usegalaxy.org, every step mapped to an installed Galaxy tool except RSEM quantification: its community wrapper was merged but not yet installed. The agent wrote one RSEM UDT so the workflow could run now. It reproduced the wrapper's test outputs exactly.
- **Orbit**, an AI assistant that plans and runs analyses on Galaxy, builds the tool when none fits. It writes a UDT, has Galaxy validate it, runs it on the real data, reads any failure, fixes and re-creates it, and carries on with the analysis. Each definition is saved with the analysis notebook, keyed to the exact tool that ran.

A middle ground

notebook ↔ production software **personal code ↔ community tools** **an algorithm ↔ infrastructure to run it**

usegalaxy.org is supported by NIH and NSF grants U24HG006620, U24HG010263, U24AI183870, U24CA284167, OT2OD037936 and 2419522. UDT jobs run on Jetstream2 at Indiana University (NSF OAC-2005506) through ACCESS, which is supported by NSF grants 2138259, 2138286, 2138307, 2137603 and 2138296. Foldseek UDT on usegalaxy.org: Anton Nekrutenko. Related posters: Galaxy Workflow Foundry (Chilton) · Orbit and Galaxy UDTs (Cain) · Orbit (Nekrutenko).

Request access · Docs · Source